



+



# Firestore

# Vue.js + Firebase실전 가이드

프론트엔드만으로 회원가입/로그인과 DB 다루기



## Vue.js 통합

Firebase JS SDK를  
Vue 프로젝트에 설정



## Authentication

이메일/비밀번호 기반  
회원가입 및 로그인 구현



## Cloud Firestore

NoSQL 데이터베이스  
실시간 읽기/쓰기

# 강의 개요

Vue.js와 Firebase로 구축하는 실전 프로젝트

Part 0

1 Firebase & 이번 강의에서 쓸 서비스 소개

2 Vue 프로젝트에 Firebase JS SDK 추가하기

3 Firebase Authentication  
이메일/비밀번호 회원가입 & 로그인

4 Cloud Firestore  
실시간 데이터 저장 & 조회

5 (선택) Firestore 보안 규칙 기본 패턴

## 최종 결과물

이메일 회원가입/로그인

Todo 리스트 실시간 동기화

사용자별 데이터 접근 제어

## 사전 요구사항

Vue.js 기본 문법

JavaScript(ES6+)

npm / 터미널 사용

Focus: 백엔드 서버 구축 없이 프론트엔드에서  
모든 기능을 구현합니다.

# Firebase란?

Google이 제공하는 BaaS(Backend as a Service) 플랫폼

## 서버 없는 백엔드 구축

서버를 직접 만들고 관리할 필요 없이, 인증, DB, 스토리지 등 백엔드 기능을 즉시 사용할 수 있습니다.

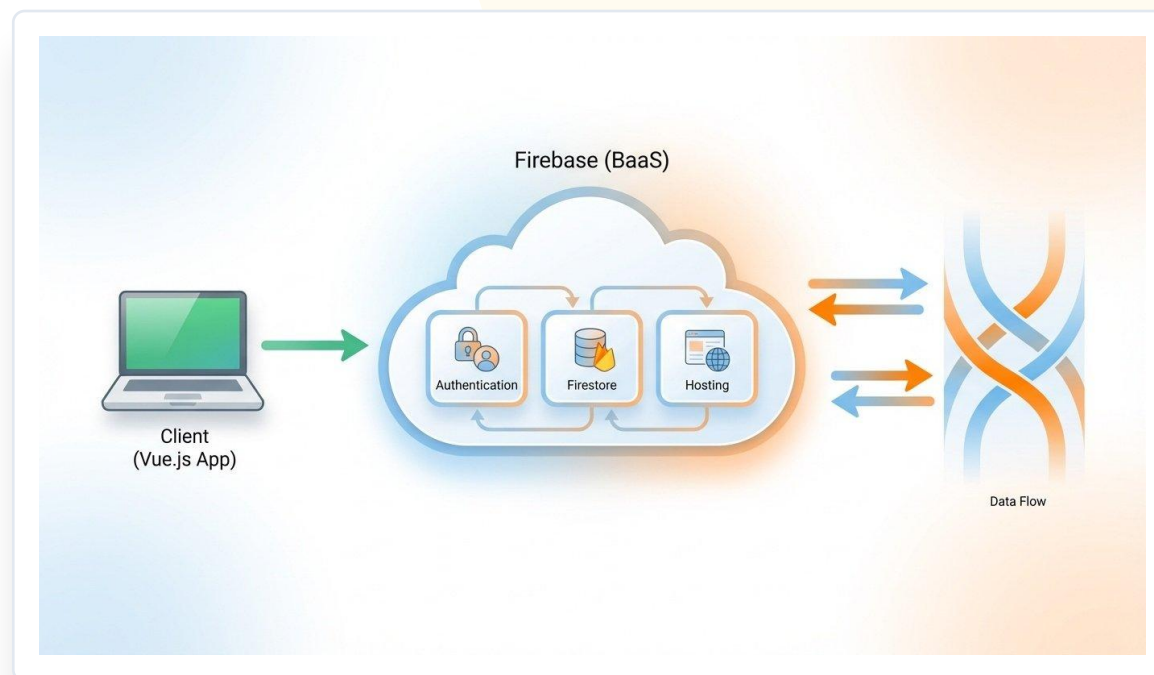
## 프론트엔드 친화적 SDK

Vue.js와 같은 프론트엔드 프레임워크에서 직접 호출할 수 있는 강력한 JavaScript SDK를 제공합니다.

## 빠른 개발 속도

복잡한 인프라 설정 없이 비즈니스 로직과 UI 구현에만 집중하여 개발 시간을 단축합니다.

"오늘 강의의 목표는 백엔드 서버 없이도 회원가입, 로그인부터 DB 읽기·쓰기까지 완전한 기능을 구현하는 것입니다."



Client(Vue) ↔ Firebase 서비스 간 직접 통신 구조

# 이번 수업에서 사용할 Firebase 기능



## Firebase Authentication

사용자 인증 및 관리

이메일/비밀번호 기반의 간편한 회원가입 & 로그인 구현

로그인 상태 실시간 추적 함수 제공 (예: `onAuthStateChanged`)



## Cloud Firestore

NoSQL 클라우드 데이터베이스

문서(Document) 기반의 유연한 NoSQL 데이터 구조

데이터 변경 사항을 즉시 반영하는 실시간 리스너 지원 (예: `onSnapshot`)

## Firebase JS SDK

(Modular API)

최신 v9+ SDK는 Tree-shaking을 지원하여 필요한 기능만 가볍게 가져올 수 있습니다. 예: `import { getAuth } from 'firebase/auth'`

```
npm install firebase
```

# Firebase 프로젝트 & Web 앱 등록

Firebase 서비스를 사용하기 위한 프로젝트 생성 및 설정 단계



## 프로젝트 만들기

Firebase Console에 접속하여  
**[프로젝트 만들기]**를 클릭하고  
이름을 설정합니다.



## 웹 앱 추가

프로젝트 개요 페이지에서  
**웹(</>) 아이콘**을 클릭하여  
앱 닉네임을 등록합니다.



## 설정 복사

생성된 firebaseConfig 객체를  
복사하여 프로젝트에 준비합니다.

```
const firebaseConfig = {  
  apiKey: "...",  
  authDomain: "...",  
  ...  
};
```

### 보안 Tip: 환경 변수 사용 권장

실제 배포 시에는 API Key 등의 정보를 코드에 직접 노출하지 말고, .env 파일을 사용하여 환경 변수로 관리하는 것이 안전합니다. (예: VITE\_FIREBASE\_API\_KEY)

# SDK 설치 및 초기화

Vue 프로젝트에 Firebase 라이브러리 연결하기

npm package: firebase

## 1 패키지 설치



```
$ npm install firebase
```

Vue 프로젝트 터미널에서 명령어를 실행하여 최신 버전(v9+)의 Firebase SDK를 설치합니다.

JS src/firebase.js

```
import { initializeApp } from 'firebase/app'
import { getAuth } from 'firebase/auth'
import { getFirestore } from 'firebase/firestore'
```

// 1. Firebase 콘솔에서 복사한 설정 객체

```
const firebaseConfig = {
  apiKey: 'YOUR_API_KEY',
  authDomain: 'PROJECT_ID.firebaseio.com',
  projectId: 'YOUR_PROJECT_ID',
  storageBucket: 'PROJECT_ID.appspot.com',
  messagingSenderId: 'YOUR_SENDER_ID',
  appId: 'YOUR_APP_ID'
}
```

// 2. Firebase App 초기화

```
const app = initializeApp(firebaseConfig)
```

// 3. 각 서비스 인스턴스 생성 및 export

```
const auth = getAuth(app)
```

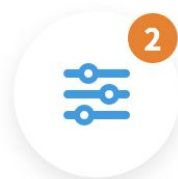
# Email/Password 인증 활성화

Firebase 콘솔에서 이메일 로그인 방식을 설정하는 과정



## Authentication 메뉴 진입

Firebase 콘솔 좌측 메뉴의  
**빌드(Build)** 섹션에서  
**Authentication**을 선택합니다.



## Sign-in method 선택

상단 탭 메뉴 중  
**Sign-in method**를 클릭하여  
제공자 목록을 확인합니다.



## 제공자 활성화

**이메일/비밀번호**를 클릭하고  
'사용 설정' 스위치를 켜 후  
**저장(Save)**합니다.

사용 설정 ☒

### 테스트 팁: 사용자 미리 만들기

Users 탭에서 [사용자 추가] 버튼을 통해 테스트용 이메일/비밀번호 계정을 미리 생성해두면, 회원가입 기능을 구현하기 전에도 로그인 테스트를 바로 진행할 수 있습니다.

# Auth 핵심 함수 정리

Firebase JS SDK (Modular) 주요 메서드

Part 3



## `getAuth(app)`

Firebase App에서 Authentication 인스턴스를 초기화하고 가져옵니다.



## `createUserWithEmailAndPassword(...)`

이메일과 비밀번호로 새로운 사용자 계정을 생성합니다.



## `signInWithEmailAndPassword(...)`

기존 계정으로 로그인하고 인증 토큰을 받아옵니다.



## `signOut(auth)`

현재 로그인된 사용자를 로그아웃 처리합니다.



## `onAuthStateChanged(auth, callback)`

로그인/로그아웃 상태 변화를 실시간으로 감지(Observer)합니다.

### 핵심 포인트

모든 인증 로직은 클라이언트(브라우저)에서 직접 Firebase 서버로 요청됩니다.

복잡한 토큰 발급 및 갱신 과정을 SDK가 자동으로 처리해줍니다.

비밀번호는 암호화되어 전송되며, 서버에 평문으로 저장되지 않습니다.

### 개발 팁

`onAuthStateChanged`는 앱이 새로고침되어도 로그인 상태를 유지(Persistence)하는 핵심 역할을 합니다. 앱 초기화 시점(mounted)에 반드시 등록하세요.

# 프로젝트 구조 및 Router 설정

Vue Router를 이용한 화면 전환(Login / Register / Main) 구성 (npm install vue-router)

Structure

## 전체 폴더 구조 (src/)

```
src/  
├── firebase.js  
├── router/  
│   └── index.js  
├── views/  
│   ├── LoginView.vue  
│   ├── RegisterView.vue  
│   └── MainView.vue  
├── components/  
│   └── TodoList.vue  
└── App.vue
```

src/router/index.js

```
import { createRouter, createWebHistory } from 'vue-router'  
  
// 화면 컴포넌트 Import  
import LoginView from '../views/LoginView.vue'  
import RegisterView from '../views/RegisterView.vue'  
import MainView from '../views/MainView.vue'  
  
const routes = [  
  {  
    path: '/',  
    name: 'login',  
    component: LoginView  
  },  
  {  
    path: '/register',  
    name: 'register',  
    component: RegisterView  
  },  
  {  
    path: '/main',
```

# Bootstrap 설치 및 main.js 설정

UI 프레임워크 및 아이콘 라이브러리 연결

npm install

## 1 패키지 설치



```
$npm install bootstrap bootstrap-icons
```

터미널에서 Bootstrap CSS 프레임워크와 전용 아이콘 패키지를 설치합니다.

## 사용 준비 완료

main.js에서 한 번만 import하면, 프로젝트 내의 모든 Vue 컴포넌트에서 별도의 선언 없이 Bootstrap 클래스(예: btn-primary)와 아이콘을 바로 사용할 수 있습니다.

src/main.js

```
import { createApp } from 'vue'
import './style.css'
import App from './App.vue'
import router from './router/index'

import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap-icons/font/bootstrap-icons.css'

const app = createApp(App)
app.use(router) // 라우터 사용 등록

app.mount('#app')
```

# LoginView.vue

Bootstrap UI

src / views / LoginView.vue

LoginView.vue

```
<script setup>
import { ref } from 'vue'
import { signInWithEmailAndPassword } from 'firebase/auth'
import { auth } from '../firebase'
import { useRouter } from 'vue-router'

const email = ref('')
const password = ref('')
const errorMessage = ref('')
const errorMessage: Ref<string, string>
const errorMessage = ref('')
errorMessage.value = ''
try {
  await signInWithEmailAndPassword(auth, email.value, password.value)
  router.push('/main')
} catch (err) {
  errorMessage.value = '로그인 실패: ' + err.code
}
</script>
```

## 로그인

이메일

비밀번호

로그인

아직 회원이 아니신가요? [가입하기](#)

## Bootstrap & Router

card, form-control 등의 Bootstrap 클래스로 별도 CSS 없이 깔끔한 UI를 구성했습니다. 로그인 성공 시 router.push('/main')을 호출하여 메인 화면으로 부드럽게 전환됩니다.

# 예제: RegisterView.vue

Bootstrap UI + Auth

src / views / RegisterView.vue

```
RegisterView.vue

const email = ref('')
const password = ref('')
const passwordConfirm = ref('')
const errorMessage = ref('')
const message = ref('')

const router = useRouter()

ny-firebase-app/src/App.vue async () => {
  errorMessage.value = ''
  message.value = ''

  if (password.value !== passwordConfirm.value) {
    errorMessage.value = '비밀번호가 일치하지 않습니다.'
    return
  }

  try {
    await createUserWithEmailAndPassword(auth, email.value, password.value)
    message.value = '회원가입이 완료되었습니다!'
    setTimeout(() => router.push('/'), 1200)
  } catch (err) {
    errorMessage.value = '회원가입 실패: ' + err.code
  }
}
```

## 회원가입

이메일

비밀번호

비밀번호 확인

회원가입

[로그인 화면으로 돌아가기](#)

## 회원가입 프로세스

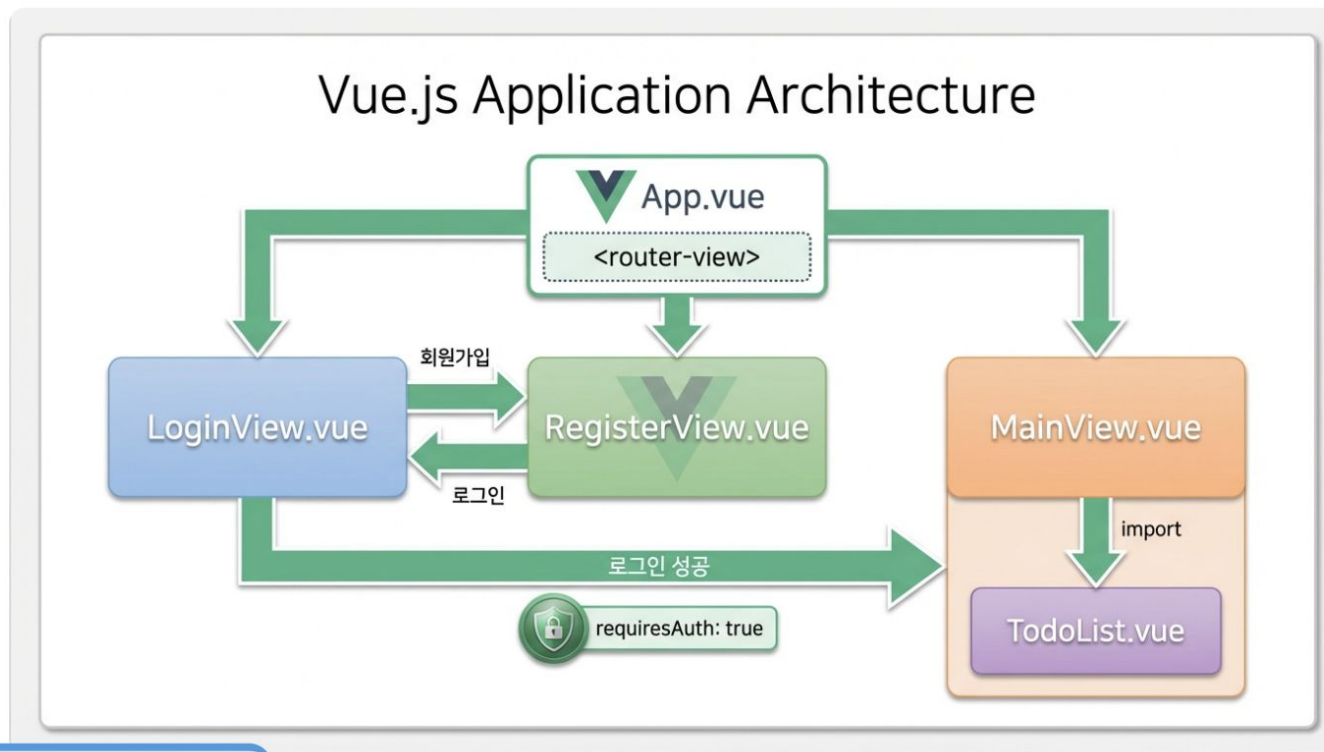
Bootstrap card 클래스로 깔끔한 폼을 구성하고, passwordConfirm으로 비밀번호 일치 여부를 검증합니다. 성공 시 router.push('/')로 로그인 화면으로 이동합니다.

# MainView.vue

Bootstrap UI

src / views / MainView.vue

container와 d-flex 등의 Bootstrap 유틸리티 클래스로 레이아웃을 쉽게 구성했습니다. 로그아웃 시 `signOut(auth)`를 호출하여 세션을 종료하고 초기 화면으로 이동시킵니다.



## Routing Flow

**LoginView**와 **RegisterView**는 자유롭게 이동 가능합니다.

로그인 성공 시에만 `router.push('/main')` 을 통해 메인 화면으로 진입합니다.



## Auth Guard

**MainView**는 `requiresAuth: true` 메타 필드로 보호됩니다.

Router의 `beforeEach` 가드에서 비로그인 사용자의 접근을 차단합니다.



## Component Tree

**MainView**는 레이아웃과 인증 체크만 담당합니다. 실제 할 일 관리 기능은 하위 컴포넌트인 **TodoList**에 위임하여 역할을 분리합니다.

# onAuthStateChanged 구독 해제 패턴

메모리 누수 방지를 위한 리스너 해제(Unsubscribe) 처리 방법



OFFICIAL DOCS

## Firebase 원리

```
// 공식 문서 예제 중 (Service Worker)

const unsubscribe = onAuthStateChanged(
  auth,
  user => { ... }
)

// 리스너 해제 (구독 취소)
unsubscribe()
```

### ■ 반환값의 비밀

onAuthStateChanged 함수를 실행하면, 해당 리스너를 해제 (Unsubscribe)할 수 있는 함수가 반환됩니다. 이를 호출하면 더 이상 상태 변화를 감지하지 않습니다.



FRAMEWORK INTEGRATION

## Vue 응용 패턴

```
let unsubscribe = null

onMounted(() => {
  // 반환된 해제 함수를 변수에 저장
  unsubscribe = onAuthStateChanged( ... )
})

onUnmounted(() => {
  // 컴포넌트가 사라질 때 구독 해제
  if (unsubscribe) unsubscribe()
})
```

### 🧩 라이프사이클 연동

SPA에서는 컴포넌트가 사라져도 리스너가 남아있을 수 있습니다. onUnmounted에서 해제 함수를 호출하여 메모리 누수를 방지해야 합니다.

# Firestore 기본 개념

## NoSQL 문서(Document) 기반 데이터베이스 구조의 이해



### Document (문서)

Key-Value 쌍으로 이루어진 JSON 유사 객체입니다. 실제 데이터가 저장되는 최소 단위입니다.



### Collection (컬렉션)

문서들의 그룹(컨테이너)입니다. 예: users, todos. 문서는 반드시 컬렉션 안에 포함되어야 합니다.



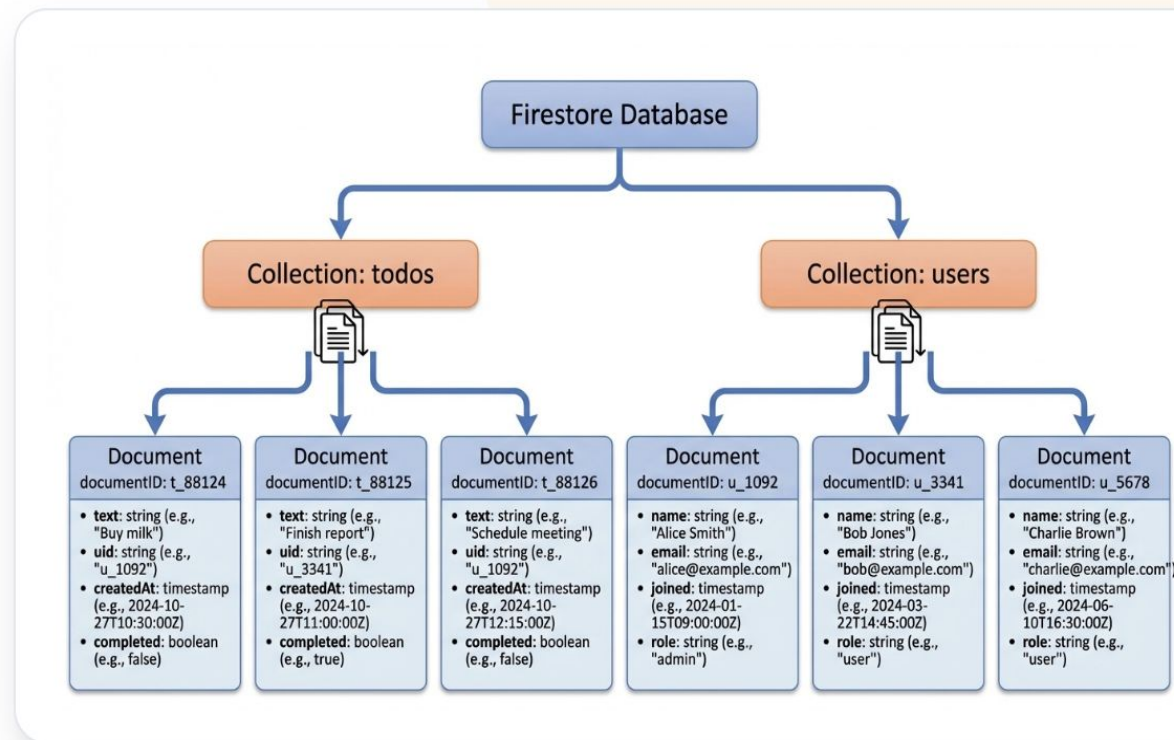
### 자동 생성 (Schemaless)

별도의 테이블 생성 과정 없이, 데이터가 처음 쓰일 때 컬렉션과 문서가 자동으로 생성됩니다.

```
import { getFirestore } from 'firebase/firestore'

const db = getFirestore(app)

// 1. 앱 초기화 후 Firestore 인스턴스를 생성합니다.
```



🗂 컬렉션 안에 문서가 들어가는 계층적 트리 구조

# Firestore 활성화 및 초기 설정

실습 전 필수! Firebase 콘솔에서 Firestore Database 설정하기

1

## 데이터베이스 생성

콘솔 좌측 메뉴 Build → Firestore Database 클릭 후, Create database 버튼을 누릅니다.

2

## 위치(Location) 선택

서비스 지역과 가까운 리전을 선택합니다. (예: 서울 asia-northeast3 또는 도쿄 asia-northeast1)

3

## 시작 모드 선택 중요

개발/수업용으로는 반드시 Test mode를 선택하세요. (기본적으로 30일간 누구나 읽기/쓰기 허용)

데이터

규칙

색인

재해 복구

사용량

Extensions



오늘 • 6:09 오후



오늘 • 6:07 오후

```
1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4     match /todos/{docId} {
5       allow read, write: if request.auth != null;
6     }
7   }
8 }
9
```

## 보안 주의사항

Test mode는 개발 편의를 위해 모든 접근을 허용합니다. 실서비스 배포 전에는 반드시 인증된 사용자만 접근 가능하도록 규칙을 수정해야 합니다.

# Firestore 주요 함수

데이터 읽기/쓰기 및 실시간 동기화

## ☰ 핵심 함수 (Modular API)

- collection** 특정 컬렉션의 위치 참조를 생성합니다.
- addDoc** 자동 생성된 ID로 문서를 컬렉션에 추가합니다.
- getDocs** 쿼리 결과나 컬렉션의 데이터를 **1회** 가져옵니다.
- onSnapshot** 데이터 변경을 **실시간**으로 감지하여 콜백을 실행합니다.
- query** 조건(where)이나 정렬(orderBy)을 위한 쿼리를 구성합니다.

JS src/components/ToDoList.vue

```
import {
  collection, addDoc, getDocs,
  query, orderBy, onSnapshot, serverTimestamp
} from 'firebase/firestore'
import { db } from '../firebase'

// 1. 컬렉션 참조 생성
const todosRef = collection(db, 'todos')

// 2. 데이터 쓰기 (자동 ID)
await addDoc(todosRef, {
  text: '새 할 일',
  uid: 'user_123',
  createdAt: serverTimestamp() // 서버 시간 사용
})

// 3. 데이터 조회 (1회성)
const q = query(todosRef, orderBy('createdAt', 'desc'))
const snap = await getDocs(q)
```

## Firestore 실시간 연동

src / components / TodoList.vue

```
const unsubscribeMyTodos = (uid) => {
  console.log('[Firestore] todos(내 것만, 정렬 없음) 구독 시작, uid =', uid)

  const todosRef = collection(db, 'todos')
  const q = query(
    todosRef,
    where('uid', '==', uid)
  )

  unsubscribeTodos = onSnapshot(
    q,
    (snapshot) => {
      isLoading.value = false
      loadError.value = ''

      // 정렬은 클라이언트에서 (선택)
      todos.value = snapshot.docs
        .map(doc => ({
          id: doc.id,
          ...doc.data()
        })))
    }
  )
}
```

The screenshot displays a web application interface. At the top, there is a navigation bar with a logo on the left and a '로그아웃' (Logout) button on the right. Below the navigation bar, a light blue banner indicates the logged-in user: '로그인한 사용자: a@a.com'. The main content area is titled 'Todo 목록 (Firestore)' and contains a form for adding new todos. The form has a text input field with the placeholder '할 일을 입력하세요' and a blue '추가' (Add) button. Below the form, a list of existing todos is shown, each with a text input field and a timestamp in a grey bubble. The todos are: 'gggggggggggggggggggggggggggggg' (timestamp: 2025. 11. 30. 오후 6:20:07), 'hello world' (timestamp: 2025. 11. 30. 오후 6:20:00), and 'ㄹㄹ' (timestamp: 2025. 11. 30. 오후 6:09:49). A red hand-drawn line highlights the entire main content area.

## 실시간 데이터 동기화

onSnapshot을 사용하면 DB 변경 시 콜백이 자동 호출되어 새로고침 없이 화면이 갱신됩니다..

# TodoList.vue Firestore 핵심 로직

실시간 리스너(onSnapshot)와 데이터 추가 패턴 분석

Logic

```
const subscribeMyTodos = (uid) => {
  console.log('[Firestore] todos(내 것만, 정렬 없음) 구독 시작, uid =', uid)

  const todosRef = collection(db, 'todos')
  const q = query(
    todosRef,
    where('uid', '==', uid)
  )

  unsubscribeTodos = onSnapshot(
    q,
    (snapshot) => {
      isLoading.value = false
      loadError.value = ''

      // ▼ 정렬은 클라이언트에서(선택)
      todos.value = snapshot.docs
        .map(doc => ({
          id: doc.id,
          ...doc.data()
        }))
    })
  )
}
```



## 실시간 동기화 (onSnapshot)

단순 1회성 조회(getDocs)가 아니라, 데이터가 변경될 때마다 **자동으로 콜백 함수가 실행**되어 화면을 최신 상태로 유지합니다.



## 단방향 데이터 흐름

데이터 추가 시 `todos.value` 를 직접 수정하지 않습니다. 서버에 저장 완료되면 리스너가 이를 감지하여 목록을 갱신하는 **"Server Source of Truth"** 패턴입니다.



## 상태 관리 (UX)

`isLoading` (초기 로딩), `isAdding` (추가 중 버튼 비활성화), `loadError` 등 상태 변수를 사용하여 사용자에게 명확한 피드백을 제공합니다.

# TodoList.vue 실행 결과 DEMO

Bootstrap UI + Firestore 실시간 동기화 구현 화면

http://localhost:5173/main

## Todo 목록 (Firestore)

할 일을 입력하세요 추가

|                |                          |
|----------------|--------------------------|
| Vue.js 강의 준비하기 | 2025. 11. 30. 오후 2:15:23 |
| Firebase 문서 읽기 | 2025. 11. 30. 오후 3:45:10 |



### 입력 폼 (Add)

Bootstrap form-control을 사용한 깔끔한 입력창과 비동기 처리 중 버튼 비활성화 상태를 제공합니다.



### 실시간 목록

Firestore onSnapshot을 통해 데이터 변경 시 자동으로 UI 리스트가 새로고침 없이 갱신됩니다.



### 타임스탬프

serverTimestamp()로 저장된 서버 시간을 포매팅하여 각 항목 우측에 배지로 표시합니다.



### 상태 피드백

로딩 중(isLoading), 저장 성공/실패 메시지를 조건부 렌더링(v-if)하여 사용자 경험을 향상시킵니다.

# Firestore 보안 규칙 (Security Rules)

데이터베이스 접근 제어 및 보안 설정 기본 패턴

Access Control

## 1 기본 보안 원칙

클라이언트 사이드 코드만으로는 데이터베이스를 보호할 수 없습니다. **Firestore Rules**를 사용하여 서버 측에서 접근 권한을 검증해야 합니다.



```
$ firebase deploy --only firestore:rules
```

\* 로컬에서 작성 후 배포하거나, Firebase Console에서 직접 수정 가능

## 2 인증 확인 패턴

**request.auth** 객체는 인증된 사용자의 정보를 담고 있습니다. 로그인하지 않은 사용자는 **request.auth** 값이 null입니다.

 firestore.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // 모든 문서에 대한 기본 규칙 (테스트용)
    // match /{document=**} {
    //   allow read, write: if true; <-- 위험: 누구나 접근 가능
    // }

    // 'todos' 컬렉션: 로그인한 사용자만 접근 허용
    match /todos/{docId} {
      allow read, write: if request.auth != null;
    }
  }
}
```

# 정리 및 다음 단계

Vue.js + Firebase 프로젝트 통합 과정 요약



## 오늘 학습한 내용

- ✓ **Firebase 모듈러 SDK** 설치 및 초기화 (v9+)
- ✓ **Authentication**: 이메일 회원가입/로그인 구현
- ✓ **상태 감지**: `onAuthStateChanged`로 로그인 유지
- ✓ **Firestore DB**: 컬렉션/문서 구조 이해 및 데이터 쓰기
- ✓ **실시간 동기화**: `onSnapshot`으로 데이터 변경 즉시 반영
- ✓ **보안 규칙**: 로그인 사용자 접근 제한 패턴 적용



## 다음 학습 단계 (Next Level)



### 전역 상태 관리 (Pinia)

User 객체와 Todo 데이터를 Pinia 스토어로 관리하여 구조화



### 보안 규칙 세분화

본인이 작성한 글만 수정/삭제 가능하도록 `request.auth.uid` 검증



### 배포 (Deployment)

Firebase Hosting으로 정적 웹사이트 배포 및 도메인 연결



### 환경 변수 분리 (.env)

API Key 등을 `.env` 파일로 관리하여 보안성 강화